

Universidad San Carlos de Guatemala

Nombre: Jonathan Eduardo Fuentes García

Carnet: 202408977

Curso: Lenguajes Formales y de Programación

Auxiliar Carlos Javier Cox Bautista

Tema:



MANUAL DE USUARIO

Fecha: 4/05/2026

Primer semestre

Introducción

TaskScript es una herramienta de escritorio que te permite escribir y analizar tableros Kanban usando un lenguaje de texto llamado TaskScript.

Con este programa puedes:

- Escribir tus tableros y tareas en un formato sencillo.
- Verificar automáticamente si el archivo está bien escrito.
- Detectar errores de escritura o de estructura.
- Obtener un diagrama del árbol de análisis que muestra cómo se interpretó tu archivo.

Este manual propone guiar paso a paso para que puedas instalar, compilar y usar la aplicación sin dificultad. :)

Índice

Índice	3
1. Requisitos e instalación	4
1.1 ¿Qué necesito?.....	4
1.2 ¿Cómo compilo el programa?	4
2. Interfaz de la aplicación	4
3. Cómo cargar o escribir un archivo .task	5
4. Ejemplos de archivos .task	6
5. Realizar el análisis.....	6
6. Interpretar los resultados	7
6.1 Tabla de Tokens	7
6.2 Tabla de Errores	8
7. Reportes generados	8
8. Árbol de derivación (Graphviz)	10

1. Requisitos e instalación

1.1 ¿Qué necesito?

- **Qt Creator** versión 5.15 o superior (recomendado Qt 6.x)
- Un compilador con soporte C++17 (g++, MinGW, MSVC 2019/2022)
- **Graphviz** (opcional, solo si quieres ver el árbol en tu computadora; también puedes usar GraphvizOnline)

1.2 ¿Cómo compilo el programa?

1. Abre Qt Creator.
2. Haz clic en File → Open File or Project... y selecciona el archivo CMakeLists.txt o el archivo .pro del proyecto.
3. Cuando te pregunte por el kit de compilación, elige el que tengas instalado (Desktop Qt 6.x o similar).
4. Haz clic en el botón Run (martillo verde) o presiona Ctrl+R.

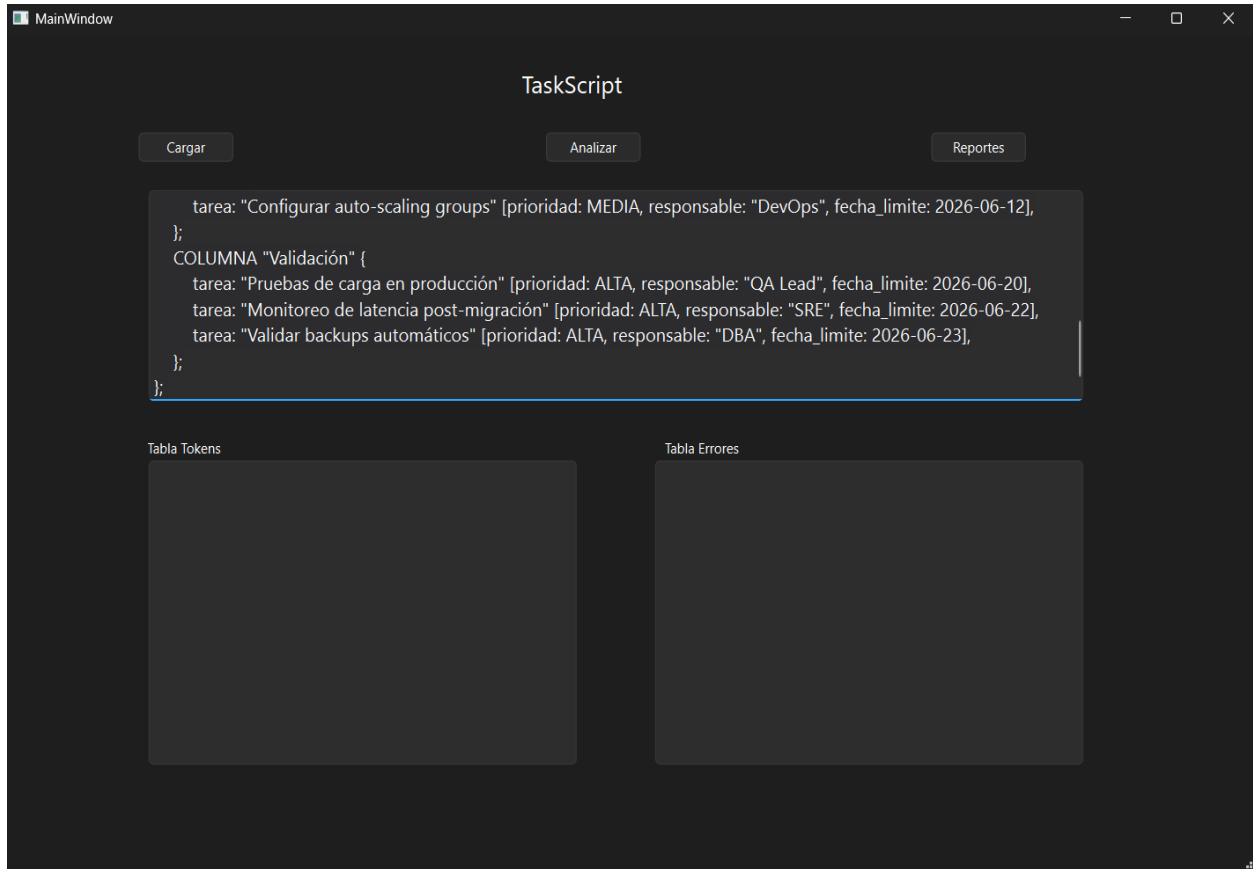
El programa se compilará y se abrirá la ventana principal.

2. Interfaz de la aplicación

La ventana principal tiene cuatro zonas importantes:

1. **Editor de código:** gran cuadro de texto donde puedes escribir o pegar el contenido de un archivo .task.
2. **Botones:**
 - **Cargar Archivo:** abre un cuadro de diálogo para buscar un archivo .task.
 - **Analizar:** ejecuta el análisis léxico y sintáctico sobre el texto actual.
 - **Generar Reportes:** repite el análisis y fuerza la creación de los reportes.

3. **Tabla de Tokens:** muestra cada palabra o símbolo encontrado, con su número, lexema, tipo y posición.
4. **Tabla de Errores:** muestra todos los errores léxicos y sintácticos encontrados, con su descripción y ubicación.



3. Cómo cargar o escribir un archivo .task

Puedes escribir el código directamente en el editor, o cargar un archivo desde tu computadora:

- Para **cargar**: presiona Cargar Archivo, selecciona un archivo con extensión .task y se mostrará en el editor.
- Para escribir: simplemente teclea o pega el código dentro del editor.

4. Ejemplos de archivos .task

Ejemplo 1: tablero con dos columnas

```
TABLERO "Mi Proecto" {  
  COLUMNA "Pendiente" {  
    tarea: "Revisar requisitos" [prioridad: ALTA, responsable: "Carlos", fecha_limite: 2026-06-01]  
  };  
  COLUMNA "Terminado" {  
    tarea: "Configurar repositorio" [prioridad: MEDIA, responsable: "Diana", fecha_limite: 2026-05-20]  
  };  
};
```

Ejemplo 2: tablero con una sola columna y atributo mínimo

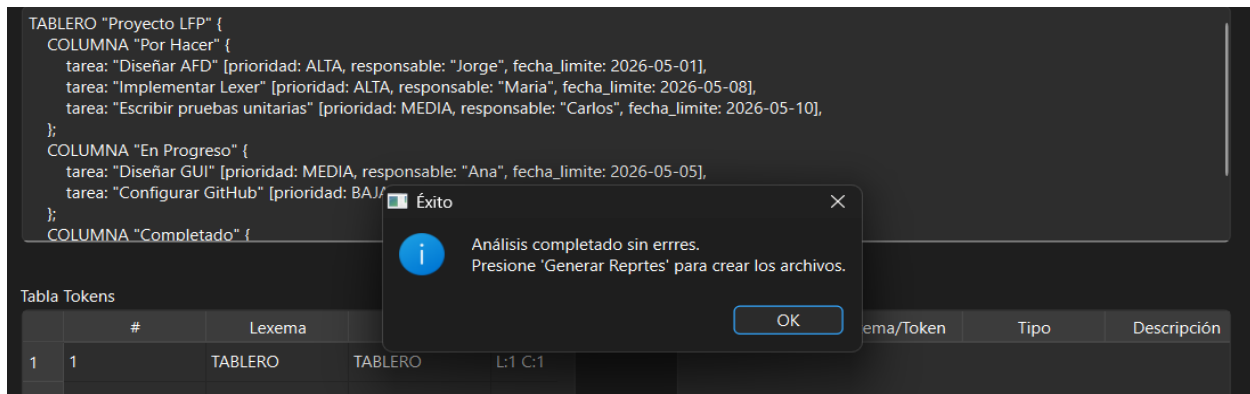
```
TABLERO "Test" {  
  COLUMNA "To Do" {  
    tarea: "T1" [prioridad: ALTA]  
  };  
};
```

Ambos ejemplos son válidos y no generan errores.

5. Realizar el análisis

Una vez que el editor contiene el código del tablero:

1. Presiona el botón **Analizar**.
2. Las tablas se llenarán automáticamente:
 - La tabla de tokens muestra todos los elementos reconocidos.
 - La tabla de errores muestra los problemas encontrados (si los hay).
3. Al final, aparecerá un mensaje indicando si el análisis fue exitoso o si se encontraron errores.



6. Interpretar los resultados

6.1 Tabla de Tokens

Cada fila muestra:

#	Lexema	Tipo	Línea:Col
1	TABLERO	TABLERO	L:1 C:1
2	"Mi Proyecto"	CADENA	L:1 C:9
3	{	LLAVE_IZQ	L:1 C:24

- **#**: número consecutivo del token.
- **Lexema**: el texto exacto encontrado en el archivo.
- **Tipo**: la categoría del token (TABLERO, COLUMNA, tarea, prioridad, CADENA, FECHA, LLAVE_IZQ, CORCHETE_DER, DOS_PUNTOS, COMA, etc.).
- **Línea:Col**: ubicación del token en el archivo original.

	#	Lexema	Tipo	Línea
1	1	TABLERO	TABLERO	L:1 C:1
2	2	"Proyecto LFP"	CADENA	L:1 C:9
3	3	{	LLAVE_IZQ	L:1 C:24
4	4	COLUMNA	COLUMNA	L:2 C:5
5	5	"Por Hacer"	CADENA	L:2 C:13
6	6	{	LLAVE_IZQ	L:2 C:25
7	7	tarea	tarea	L:3 C:9
-	-	-	-	-

6.2 Tabla de Errores

Si hay errores, cada fila muestra:

#	Lexema/Token	Tipo	Descripción	Línea:Col
1	"color"	Léxico	Palabra no reconocida: 'color'	L:3 C:18
2	","	Sintáctico	Se esperaba '}'	L:5 C:9

- **Tipo:** léxico (palabra desconocida, carácter inválido, cadena sin cerrar) o sintáctico (falta un símbolo, orden incorrecto).
- **Línea:Col:** lugar exacto donde se detectó el error.

Si la tabla está vacía, significa que el archivo es correcto.

Tabla Errores			
	Lexema/Token	Tipo	Descripción
1	@	Lexico	Caracter no valido dentro de cac
2	"Juan	Lexico	Cadena sin cerrar antes de fin de
3	]	Sintactico	Se esperaba CADENA

7. Reportes generados

Si el análisis termina sin errores, el programa crea automáticamente tres archivos HTML y un archivo DOT.

Reporte 1 – Tablero Kanban Visual (reporte_kanban.html)

Muestra las columnas del tablero como tarjetas de colores:

- **ALTA:** borde rojo
- **MEDIA:** borde amarillo
- **BAJA:** borde verde

Cada tarjeta contiene el nombre de la tarea, la prioridad, el responsable y la fecha límite.

Las columnas se presentan lado a lado, como un tablero Kanban real.



Reporte 2 – Carga por Responsable (reporte_carga.html)

Tabla con el resumen de tareas asignadas a cada persona:

- Total de tareas
- Cantidad de tareas ALTA, MEDIA y BAJA
- Barra de progreso con el porcentaje sobre el total del tablero

Ejemplo:

Carga de Trabajo por Responsable

Responsable	Total	Alta	Media	Baja	Distribución
Ana	1	0	1	0	16%
Carlos	1	0	1	0	16%
Jorge	1	1	0	0	16%
Maria	1	1	0	0	16%
Pedro	2	0	0	2	33%

Reporte 3 – Tareas con Fechas Límite (reporte_tareas_fechas.html)

Listado simple con todas las tareas del tablero y su fecha de entrega

Ejemplo:

Tareas y Fechas Límite

Tarea	Responsable	Fecha Límite
"Diseñar AFD"	Jorge	2026-05-01
"Implementar Lexer"	Maria	2026-05-08
"Escribir pruebas unitarias"	Carlos	2026-05-10
"Diseñar GUI"	Ana	2026-05-05
"Configurar GitHub"	Pedro	2026-04-18
"Investigar Qt"	Pedro	2026-04-20

8. Árbol de derivación (Graphviz)

Además de los reportes HTML, se genera un archivo llamado arbol.dot que contiene el árbol de derivación del análisis sintctico.

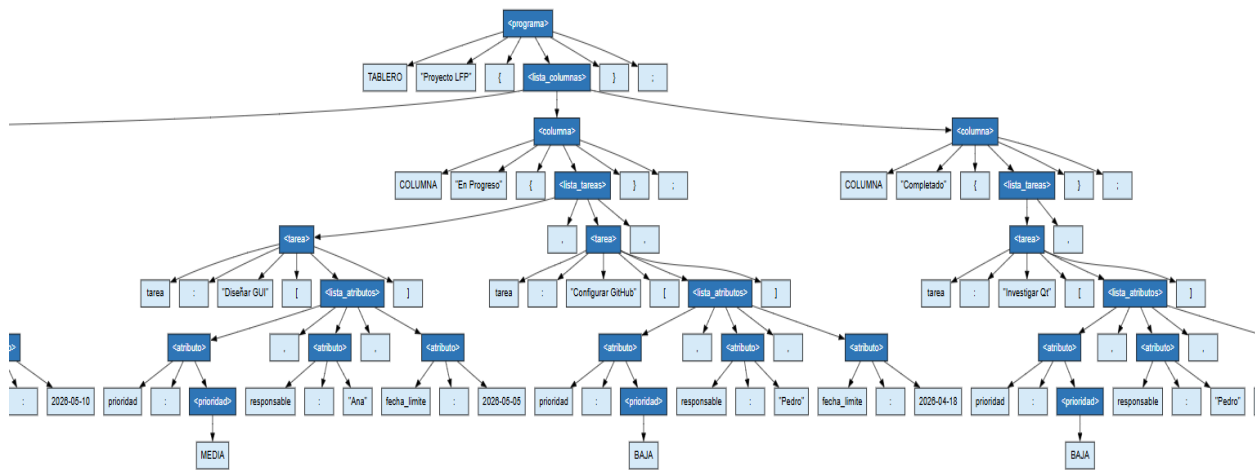
¿Cómo verlo?

1. Entra a [GraphvizOnline](#).
2. Abre el archivo arbol.dot con un bloc de notas.

3. Copia todo el contenido.
4. Pégallo en la página de GraphvizOnline.

Verás un diagrama donde:

- Los nodos azul oscuro representan las producciones de la gramática (<programa>, <columna>, etc.).
- Los nodos celeste representan los tokens concretos (TABLERO, "Mi Proyecto", {, }, etc.).



9. Solución de problemas comunes

Problema: El programa no compila y dice "No suitable kit found".

Solución: Ve a Edit → Preferences → Kits y selecciona un kit con un compilador C++17 válido.

Problema: Al cargar un archivo .task, aparece un mensaje de "Error de apertura".

Solución: Asegúrate de que el archivo existe y que la extensión es .task. Prueba con uno de los ejemplos del manual.

Problema: Al hacer clic en "Analizar" no pasa nada.

Solución: Revisa que el editor no esté vacío. Si cargaste un archivo, verifica que se muestre en el editor antes de analizar.

Problema: No se generan los reportes.

Solución: Los reportes solo se crean si el análisis no tiene errores. Corrige los errores indicados en la tabla de errores y vuelve a analizar.